ROTEIRO DE EXECUÇÃO / ETAPA 01

A entrega e o recorte do projeto

Evolução do Sistema Bancário

Engenharia reversa, BDD, separação cliente/servidor e qualidade de testes.

Objetivo: produzir uma nova versão do sistema Java com frontend separado da camada de negócio, preservando comportamentos acordados e demonstrando o fluxo principal com testes automatizados.

Natureza deste documento: plano de execução baseado no PDF “refatoracao de codigo llm - processo” e na inspeção do código local em 26/10/2026. A implementação, os testes, as métricas e a publicação descritos aqui são atividades a realizar; não são resultados concluídos.

Exigência da tarefa	Como atender
Escolher cinco requisitos vinculados	RF02, RF03, RF09, RF08 e RF12, conectados pelo fluxo de cadastro, abertura de conta e transferência.
Aplicar a metodologia com IA	Extrair regras do legado, validar com os alunos, formalizar cenários e refatorar com regressão e comparação de comportamento.
Separar cliente e servidor	Frontend HTML/CSS/JavaScript consumindo API REST; backend Java com Spring Boot e serviços de negócio.
Testar o fluxo principal	Caso CT-FL01, cenário Gherkin, testes Java de API e teste E2E no navegador.
Entregar e apresentar	Código, README, contratos, evidências e declaração de IA no GitHub; demonstração para o professor.

Sequência de trabalho

Diagnosticar o legado → extrair e validar regras → escrever cenários → definir contratos → separar camadas → testar e comparar → medir cobertura e mutação → documentar e apresentar.

A autoria e a responsabilidade técnica permanecem com os alunos. O grupo deve revisar, executar, compreender e explicar cada parte entregue.

ROTEIRO DE EXECUÇÃO / ETAPA 02

Cinco requisitos, um fluxo de negócio

Requisito	Regra e vínculo	Evidência atual
RF02 - Cadastrar cliente	Cadastrar cliente com ID único; utiliza a validação de idade de RF03.	GerenciadoraClientes.cadastraCliente ; TDD/RF02.java.
RF03 - Validar idade	O código aceita de 18 a 65 anos, inclusive. É uma condição do cadastro.	GerenciadoraClientes.validaIdade; TDD/RF03.java testa apenas 17 anos.
RF09 - Cadastrar conta	Exige cliente existente e ID da conta único; permite seguir para consulta e movimentação.	GerenciadoraContas.cadastraContaP araCliente; TDD/RF09.java.
RF08 - Consultar conta	Retorna conta por ID; retorna null se ausente. Permite verificar saldos antes e depois de RF12.	GerenciadoraContas.pesquisaConta; BDD/RF08.java.
RF12 - Transferir valores	Valor maior que zero e saldo suficiente; debita origem e credita destino.	GerenciadoraContas.transfereValor; TDD/RF12.java.

Por que estes cinco estão relacionados?

RF03 valida os dados usados por RF02. O cliente criado permite RF09. RF08 mostra as contas criadas e seus saldos. RF12 movimenta essas contas, e uma nova consulta comprova seu resultado. O mesmo fluxo é repetido para dois clientes, formando origem e destino.

Limites do recorte

Ativação, desativação e remoção ficam fora da demonstração principal. Seus testes existentes devem ser preservados se as classes compartilhadas forem alteradas. Não adicionar depósito, autenticação ou banco de dados persistente como requisitos obrigatórios desta entrega.

Dados do exemplo

Cliente Ana: ID 101, 25 anos, conta 1001. Cliente Bruno: ID 102, 30 anos, conta 1002. As contas são abertas com saldo zero. Para testar transferência, uma fixture interna e exclusiva do ambiente de teste atribui R$ 500,00 e R$ 100,00 após a abertura; isso não representa uma funcionalidade de depósito.

ROTEIRO DE EXECUÇÃO / ETAPA 03

Engenharia reversa e validação humana

1. Registrar a linha de base

Criar uma branch de trabalho e registrar o commit inicial. Inventariar Main, entidades, gerenciadoras, exceções e testes. O projeto atual usa estrutura Eclipse, JavaSE-1.8 e JUnit 4.12 em .classpath; ainda não há pom.xml no inventário inspecionado.

2. Executar e caracterizar o legado

Reproduzir o build e executar os testes existentes antes de refatorar. Registrar comando, ambiente e resultado. Acrescentar testes de caracterização dos cinco requisitos, principalmente o sucesso da transferência, ainda ausente em TDD/RF12.java.

3. Extrair regras por módulo

Separar a análise em cadastro, contas e transferências. Mapear condições, efeitos sobre listas e saldos, retornos e exceções. Vincular cada RN a um método e a um teste. Para este projeto pequeno, leitura estrutural por classe é suficiente; AST ou RAG só seriam necessários se a escala justificasse.

Prompt adaptado do primeiro PDF

```
Atue como especialista em requisitos e engenharia reversa.
Analise o código e os testes fornecidos, por módulo.
Descreva: visão geral, RN identificadas, entradas e saídas,
critérios Gherkin e premissas/avisos.
Para cada regra, cite classe e método de origem.
Separe comportamento observado de melhoria proposta.
Não invente regras não presentes no código.
```

4. Revisar as regras com o grupo

Cada aluno responsável deve conferir a descrição contra código e testes. Resolver divergências e registrar a decisão antes de implementar. O limite superior de 65 anos existe no código, embora o README destaque apenas a idade mínima; preservá-lo como linha de base até decisão explícita do grupo.

Artefatos e condição de saída

Produzir docs/diagnostico-legado.md, docs/regras-negocio.md e docs/decisoes.md. A etapa termina com regras rastreáveis e decisões registradas. Não afirmar que o legado está correto apenas porque seus testes passam.

ROTEIRO DE EXECUÇÃO / ETAPA 04

Regras extraídas e decisões de evolução

ID	Constatação no código	Tratamento planejado
RN01	cadastraCliente valida ID duplicado e depois idade.	Preservar unicidade e ausência de inserção quando houver rejeição.
RN02	validaIdade rejeita idade menor que 18 ou maior que 65.	Preservar 18 a 65 inclusive; cobrir 17, 18, 65 e 66.
RN03	Cadastro de conta rejeita cliente ausente e ID de conta duplicado.	Manter validações no backend e padronizar resposta REST.
RN04	pesquisaConta retorna conta ou null.	Manter significado; adaptar ausência para HTTP 404 na API.
RN05	Transferência exige valor positivo e saldo suficiente.	Preservar resultados e validar os dois saldos, inclusive em rejeição.

Pontos que precisam de decisão e testes explícitos

Vínculo cliente/conta: cadastraContaParaCliente recebe o cliente, mas não atualiza Cliente.idContaCorrente nem mantém associação na conta. Proposta: cadastrar o cliente sem conta vinculada e efetivar o vínculo no serviço de abertura; validar a política de uma conta por cliente e a representação de “sem conta”.

Fluxo duplicado: Main cadastra cliente e conta juntos, chamando validadores e inserções diretamente. A nova interface terá duas ações explícitas, cadastro e abertura; a mudança deve ser registrada, mantendo as regras de negócio acordadas.

Contas ausentes: transfereValor pode acessar uma referência nula. Proposta: rejeitar com erro controlado antes de alterar qualquer saldo.

Dinheiro e atomicidade: o legado usa double e atualiza destino antes da origem. Proposta de evolução: BigDecimal, escala de duas casas, rejeição de valor com fração de centavo e operação indivisível. Num backend em memória e instância única, proteger a operação inteira com o mesmo bloqueio; se houver banco, usar transação real.

Mesma conta, conta inativa e concorrência: as regras não estão suficientemente definidas para inclusão automática. Registrar decisões; qualquer restrição nova deve ser identificada como evolução e ter teste correspondente.

ROTEIRO DE EXECUÇÃO / ETAPA 05

Formalizar o comportamento com BDD

Escrever os cenários antes da nova implementação. O arquivo .feature é o contrato de comportamento; sua execução exige os steps Java e um runner Cucumber configurado. Os alunos revisam dados, resultados esperados e decisões antes de automatizar.

Cenário principal - CT-FL01

```
# language: pt
Funcionalidade: Abertura e movimentação de contas
  Como operador do sistema bancário
  Quero cadastrar clientes e movimentar suas contas
  Para conferir a transferência e os saldos resultantes

  Cenário: Transferir entre contas de clientes cadastrados
    Dado que o ambiente de teste está vazio
    Quando cadastro Ana, ID 101, com 25 anos
    E cadastro Bruno, ID 102, com 30 anos
    Então os dois clientes são cadastrados
    Quando abro a conta 1001 para o cliente 101
    E abro a conta 1002 para o cliente 102
    Então as contas existem e estão vinculadas aos clientes
    E ambas possuem saldo inicial de 0,00
    Dado que a fixture interna define os saldos
      | conta | saldo  |
      | 1001  | 500,00 |
      | 1002  | 100,00 |
    Quando consulto as duas contas
    Então vejo os saldos 500,00 e 100,00
    Quando transfiro 150,00 da conta 1001 para a conta 1002
    Então a transferência é confirmada
    Quando consulto novamente as duas contas
    Então os saldos são 350,00 e 250,00
    E a soma dos saldos permanece 600,00
```

Organização proposta

Salvar em backend/src/test/resources/features/fluxo-bancario.feature. Os steps Java exercitam a API HTTP com o servidor de teste iniciado e estado isolado por cenário. A fixture é interna ao perfil de testes, desabilitada na execução normal; não expor uma rota pública para ajustar saldos.

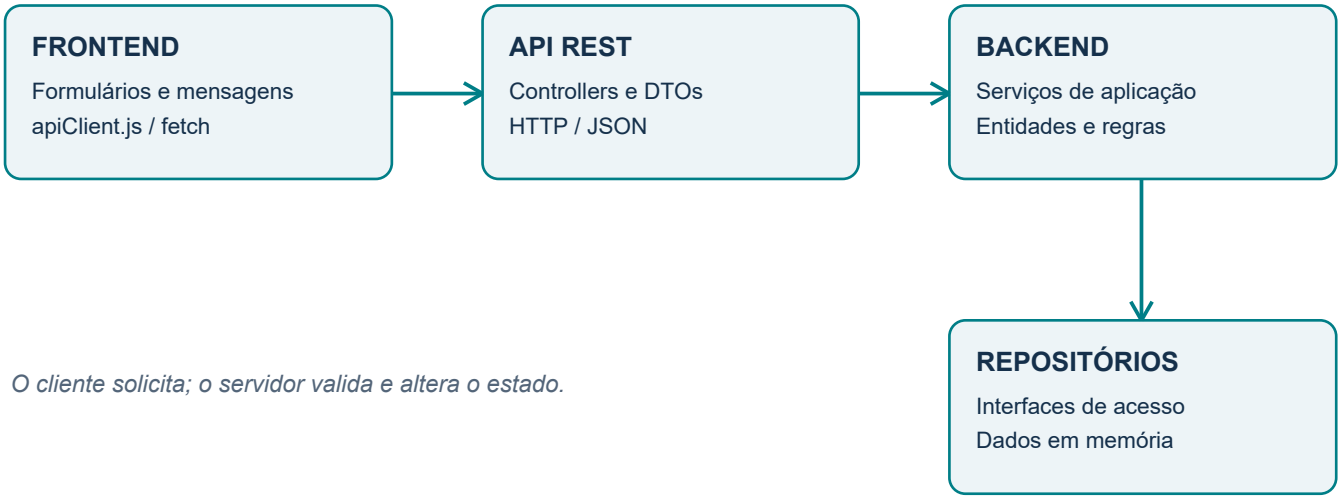
Rastreabilidade

O cadastro cobre RF02 e RF03; a abertura cobre RF09; as consultas cobrem RF08; a transferência e a confirmação dos saldos cobrem RF12. O cenário principal não substitui os testes de exceção e fronteira.

ROTEIRO DE EXECUÇÃO / ETAPA 06

Separação cliente/servidor por API REST

Solução escolhida para o roteiro: frontend Web simples, com HTML, CSS e JavaScript; backend Java com Spring Boot. A fronteira é o contrato HTTP/JSON. O frontend não importa entidades Java, não acessa listas de contas e não calcula o saldo final.



O cliente solicita; o servidor valida e altera o estado.

Camada	Responsabilidade	Exemplos de arquivos
Cliente	Coletar dados, chamar a API, exibir resultados e erros. Validação visual não substitui a do servidor.	index.html; apiClient.js; clientes.js; contas.js; transferencias.js.
API	Receber DTOs, validar formato, delegar ao serviço e converter resultados para HTTP.	ClienteController; ContaController; TransferenciaController; ApiExceptionHandler.
Aplicação e domínio	Orquestrar cadastro, vínculo e transferência; garantir regras e alteração consistente.	CadastroClienteService; AberturaContaService; TransferenciaService; Cliente; ContaCorrente.
Repositórios	Consultar e armazenar entidades. Primeiro usar implementação em memória, isolada por teste.	ClienteRepository; ContaRepository; implementações InMemory.

Estratégia de migração

Extrair de Main a orquestração de cadastro e deixá-la nos serviços. Substituir chamadas diretas da interface por apiClient. Adaptar as gerenciadoras ou migrar suas regras gradualmente, mantendo testes durante a transição. Evitar duas cópias permanentes da mesma regra.

Para simplificar a demonstração, o backend pode servir os arquivos estáticos construídos a partir de frontend/. O código-fonte continua separado e as chamadas continuam via REST. Se forem usados servidores locais distintos, configurar a URL da API e permitir CORS apenas para a origem de desenvolvimento.

ROTEIRO DE EXECUÇÃO / ETAPA 07

Contrato de comunicação do novo sistema

Os endpoints abaixo são uma proposta a implementar e documentar em docs/api.yaml. IDs são explícitos para manter rastreabilidade com o legado. Retornos booleanos e null serão traduzidos na borda da API, sem transferir as regras para o navegador.

Operação	Entrada resumida	Resposta esperada
POST /api/clientes RF02 + RF03	id, nome, idade, email. Sem conta vinculada nesta etapa.	201: cliente criado; 409: ID duplicado; 422: idade fora de 18 a 65.
POST /api/clientes/{id}/contas RF09	id da nova conta. Saldo inicial definido no servidor como 0,00.	201: conta e vínculo criados; 404: cliente ausente; 409: ID de conta duplicado.
GET /api/contas/{id} RF08	ID da conta no caminho.	200: ID, cliente vinculado, saldo e situação; 404: conta ausente.
POST /api/transferencias RF12	origemId, destinoId, valor.	200: confirmação; 404: conta ausente; 422: valor inválido ou saldo insuficiente.

Exemplo de requisição e resposta

```
POST /api/transferencias
{ "origemId": 1001, "destinoId": 1002, "valor": "150.00" }

HTTP 200
{ "origemId": 1001, "destinoId": 1002,
  "saldoOrigem": "350.00", "saldoDestino": "250.00" }

HTTP 422
{ "codigo": "SALDO_INSUFICIENTE",
  "mensagem": "Saldo insuficiente para a transferência." }
```

Decisões de contrato

Usar valores monetários decimais em strings no JSON e BigDecimal no servidor; formatar moeda apenas para exibição. Campos obrigatórios ausentes e JSON inválido retornam 400. A política de “uma conta por cliente” deve ser resolvida e adicionada ao contrato antes da implementação.

Uma operação rejeitada não pode deixar cadastro parcial ou saldos alterados. O teste deve consultar o estado depois da rejeição; verificar apenas status HTTP é insuficiente.

ROTEIRO DE EXECUÇÃO / ETAPA 08

Ferramentas e preparação do ambiente

Ferramenta	Uso no trabalho
Git + GitHub	Branch, commits por etapa, comparação com a linha de base, revisão e entrega reproduzível.
IDE com refatoração Java	IntelliJ IDEA, Eclipse ou VS Code: Extract Method/Class, Rename e Move, com compilação e regressão a cada passo.
JDK 21 + Maven Wrapper	Base proposta para o novo backend. Fixar uma versão estável compatível do Spring Boot e dependências no pom.xml; registrar no README.
Spring Boot / Spring MVC	Expor a API REST e separar controllers, serviços e repositórios; MockMvc verifica a camada HTTP sem servidor de rede [4].
JUnit Jupiter + Cucumber-JVM	JUnit para domínio e integração; Cucumber para executar Gherkin por steps Java [3]. Migrar JUnit 4 ou usar Vintage temporariamente.
JaCoCo	Instrumentação e relatórios de cobertura do código Java; preparar agente e gerar relatório no ciclo Maven [1].
PIT / PITest	Mutação das classes Java de negócio para avaliar a força dos testes; filtrar classes e testes do recorte [2].
Cypress + Node.js	Executar o fluxo real no navegador contra o backend de testes, verificando as requisições e os saldos exibidos [5].

Preparação, na ordem

1. Criar o build Maven e separar código de produção e teste. 2. Configurar os testes antigos: classes RF02, RF03 etc. não seguem o padrão usual *Test; renomeá-las ou incluir explicitamente seus padrões no Surefire. 3. Confirmar contagem e descoberta dos testes. 4. Configurar JUnit/Cucumber e PIT compatíveis; para Jupiter, incluir pitest-junit5-plugin com versão compatível. 5. Fixar versões e adicionar Maven Wrapper e package-lock.json.

Escopo das medidas

JaCoCo e PIT medem o backend Java, não o JavaScript do frontend. Cypress cobre a interação de ponta a ponta. Não usar percentuais do backend como se fossem cobertura de todo o sistema.

As escolhas são propostas técnicas para este trabalho. A configuração e os comandos das próximas páginas passam a valer depois da criação do build; não estão disponíveis no projeto atual.

ROTEIRO DE EXECUÇÃO / ETAPA 09

Implementação incremental e comparação

Passo	Ação concreta	Evidência para revisão
1 - Build e baseline	Estabelecer build reproduzível e executar o legado; preservar a versão inicial em tag após revisão das mudanças locais.	Commit de referência, lista de testes e relatório inicial.
2 - Especificação	Revisar RN01 a RN05 e decisões; criar .feature e testes antes das alterações.	Regras, critérios de aceite e testes que evidenciem lacunas.
3 - Extrair serviços	Mover a orquestração de Main para serviços, reduzir dependências e criar interfaces de repositório.	Testes do domínio aprovados a cada mudança pequena.
4 - Criar a API	Introduzir DTOs, controllers e tratamento de erros; manter regras no servidor.	Testes de contrato e integração para os cinco RF.
5 - Criar o frontend	Implementar formulários e consultas; centralizar fetch em apiClient.js.	Demonstração de cliente separado e requisições REST.
6 - Comparar versões	Executar entradas equivalentes no legado e no novo backend. Normalizar booleanos/null versus HTTP e valores monetários.	Tabela antes/depois com saída e estado final.
7 - Corrigir e medir	Tratar diferenças, rodar regressão, cobertura e mutação; investigar sobreviventes.	Relatórios e justificativas ligados ao commit avaliado.

Como fazer o teste diferencial

Usar os mesmos dados e estado inicial em cada versão: idades 17/18/65/66; IDs duplicados; abertura com cliente ausente; consulta ausente; transferência válida, nula/negativa e sem saldo. Comparar aceitação/rejeição, entidades criadas e saldos. Usar reset determinístico antes de cada execução.

Diferença esperada: null na consulta do legado vira 404 na API. Diferença de correção: conta ausente na transferência deixa de causar exceção não tratada. Registrar a decisão e o teste correspondente; não exigir igualdade com um defeito reconhecido.

Condição de saída

Cada divergência foi resolvida ou justificada, os cinco requisitos têm evidências e o frontend acessa o negócio apenas pela API. Testes que detectem falhas reais não podem ser removidos apenas para melhorar números.

ROTEIRO DE EXECUÇÃO / ETAPA 10

Caso de teste do fluxo principal

CT-FL01 - Cadastro, abertura, consulta e transferência.
Nível: cenário de sistema/E2E. Responsável pela execução: aluno designado pelo grupo. Pré-condições: backend e frontend iniciados, ambiente vazio e fixture interna de testes habilitada. Dados: os dois clientes e contas da etapa 02.

Passo	Ação	Resultado esperado
1	Cadastrar Ana (101, 25 anos) e Bruno (102, 30 anos) pelo frontend.	Dois cadastros confirmados; API retorna 201; idade passa pela validação do backend.
2	Abrir conta 1001 para Ana e 1002 para Bruno.	API retorna 201; IDs únicos, vínculos corretos e saldos zero.
3	Aplicar fixture interna após a abertura, sem botão público ou endpoint de produção.	Saldos controlados: 500,00 na origem e 100,00 no destino.
4	Consultar ambas as contas na interface.	API retorna 200; frontend mostra os valores iniciais e os titulares.
5	Transferir 150,00 de 1001 para 1002 pela interface.	Uma requisição REST é concluída com sucesso e confirmação visível.
6	Consultar novamente origem e destino.	Saldos 350,00 e 250,00; total 600,00; IDs e vínculos preservados.

Automação e oráculo

Cucumber executa o cenário de API com servidor real de teste, e Cypress repete o fluxo pelo navegador. Usar data-testid estáveis, interceptar as chamadas reais e aguardar a resposta. Não simular a transferência no E2E principal. A fixture pode ser aplicada por cy.task no ambiente de teste, com acesso interno ao estado do servidor.

Aprovar apenas se as mensagens, status, corpos de resposta e os dois saldos coincidirem com os valores esperados. Asserções numéricas do backend usam comparação decimal; no frontend, conferir também a apresentação em moeda.

Registro obrigatório da execução

Anotar commit, data, executor, versões, pré-condições, passos, esperado, obtido, status e evidência. Guardar relatório Cucumber e evidência Cypress. **Status inicial: não executado.** Não preencher “aprovado” até existir uma execução verificável.

ROTEIRO DE EXECUÇÃO / ETAPA 11

Regressão, JaCoCo e PIT

Teste	Variação a executar	Verificação essencial
CT-02 / RF03	Idades 17, 18, 65 e 66.	Rejeitar 17/66; aceitar 18/65; rejeição não insere cliente.
CT-03 / RF02	Repetir ID de cliente.	409 na API; dados e quantidade de clientes preservados.
CT-04 / RF09	Cliente ausente ou ID de conta repetido.	404 ou 409; nenhuma conta ou vínculo parcial criado.
CT-05 / RF08	Consultar ID inexistente.	404 e mensagem clara; nada alterado no repositório.
CT-06 / RF12	Valor 0, negativo, maior que o saldo e igual ao saldo.	Rejeitar inválidos sem alterar saldos; aceitar valor igual ao saldo e zerar origem.
CT-07 / RF12	Conta de origem ou destino ausente.	Erro controlado, sem alteração parcial; cobrir correção proposta.

Executar e inspecionar cobertura

Configurar JaCoCo prepare-agent, report e check no pom.xml. No recorte do domínio/serviços, propor como meta do grupo pelo menos 90% de instruções e 90% de ramificações. São metas sugeridas, não exigências do professor nem resultados obtidos. Examinar os caminhos descobertos, além do percentual [1].

```
cd backend
.\mvnw.cmd clean verify
.\mvnw.cmd test-compile org.pitest:pitest-maven:mutationCoverage
```

Relatórios previstos: target/site/jacoco/index.html e target/pit-reports/. Para JaCoCo, garantir que a JVM de teste receba o agente. Os testes Java de domínio/integração e os relatórios de UI têm escopos distintos; E2E externo não é instrumentado automaticamente [1, 2].

Interpretar mutação e fortalecer testes

Configurar PIT para entidades e serviços dos cinco RF e executar somente após a suíte normal passar. Propor meta inicial de 80% de mutação, com análise de todos os sobreviventes. Exemplos: trocar > por >= exige teste com valor zero; remover débito exige conferir saldo da origem; alterar limite etário exige testes de fronteira.

Registrar mutantes mortos, sobreviventes, sem cobertura e erros. Investigar possíveis equivalentes e justificar, sem exclusões amplas. A aceitação exige fluxo principal aprovado, regressão estável e defeitos críticos resolvidos; percentual isolado não basta.

ROTEIRO DE EXECUÇÃO / ETAPA 12

GitHub, entrega e apresentação ao professor

```
SistemaBancario/
  frontend/                # interface e apiClient
  backend/
    pom.xml, mvnw, mvnw.cmd
    src/main/java/...      # api, application, domain, infra
    src/test/java/...      # unitários, API e steps BDD
    src/test/resources/features/
  e2e/                     # Cypress e dados de teste
  docs/                    # regras, decisões e api.yaml
  evidencias/              # relatórios selecionados
  .github/workflows/ci.yml
  README.md
  USO_DE_IA.md
```

Organizar e publicar

O remoto configurado é github.com/gabrielaPLMPRO/SistemaBancario. Trabalhar em branch, revisar as alterações locais e usar commits pequenos por etapa. Abrir PR com problema, solução e validação; publicar a entrega revisada e identificar seu commit/tag. Esta tarefa de documentação não publica alterações no GitHub.

O README deve explicar objetivo, cinco RF, arquitetura, decisões, instalação, comandos exatos, dados de demonstração, execução dos testes e localização dos relatórios. A CI proposta deve executar build/testes/JaCoCo e E2E com servidor pronto; PIT pode rodar em job separado. Subir relatórios como artefatos e guardar uma amostra ligada ao commit em evidencias/. Excluir dependências, builds e segredos do versionamento.

Checklist de entrega

1. Frontend separado, sem regra bancária duplicada. 2. API e cinco RF implementados. 3. Cenário principal reproduzível e aprovado. 4. Regressão, JaCoCo e PIT com evidências e análise. 5. Comparação legado/novo e decisões documentadas. 6. GitHub acessível ao professor, instruções testadas em checkout limpo. 7. Autores, contribuições e uso de IA declarados.

Roteiro de apresentação - sugestão de 10 a 12 minutos

1 min: problema e cinco requisitos. **2 min:** regras extraídas e diferenças encontradas. **2 min:** arquitetura e chamada cliente/API/serviço. **3 min:** executar CT-FL01 e uma rejeição. **2 min:** mostrar JaCoCo e um mutante sobrevivente ou eliminado. **1 a 2 min:** GitHub, autoria, IA, limitações e decisões.

Cada aluno deve explicar uma parte que tenha revisado e executado. Preparar vídeo curto ou capturas como apoio se a demonstração ao vivo falhar. Confirmar com o professor duração e formato finais.

ROTEIRO DE EXECUÇÃO / ETAPA 13

Autoria, uso de IA e referências

Declaração a manter no repositório

“Este trabalho é de autoria dos alunos [nomes e matrículas], responsáveis pela análise, pelas decisões de projeto, pela implementação, pela execução dos testes e pela revisão da entrega. Utilizamos Codex (OpenAI) como apoio à leitura das referências, inspeção do código e elaboração deste roteiro metodológico. Outras utilizações de IA durante implementação, testes ou documentação serão registradas com finalidade e validação humana. Os resultados apresentados correspondem a execuções reais identificadas por commit e ambiente.”

Preencher os nomes e atualizar o texto para refletir o trabalho efetivamente realizado. O PDF-base declara uso de Gemini/NotebookLM na síntese das referências; isso deve ser atribuído ao material-base e não apresentado como ferramenta usada pelos alunos sem confirmação.

Registro de contribuição e evidência

Em USO_DE_IA.md, registrar data, ferramenta/modelo quando disponível, propósito, prompt ou resumo da solicitação, artefato apoiado, revisão humana, correções e responsável. Em docs/contribuicoes.md, relacionar tarefas e commits de cada aluno. Não atribuir à IA a validação de regras ou a aprovação de resultados que não foram executados.

Referências locais utilizadas

Base principal: *refatoracao de codigo llm - processo.pdf*, 6 páginas: engenharia reversa, requisitos, validação humana, BDD, reconstrução, comparação e qualidade de testes. Complementos: Oliveira (2025), *Um estudo sobre a aplicação de LLMs no teste de software*; Campos (2024), *Proposta para Implantação de Automação de Testes de Software usando BDD*. Todos estão na pasta contexto para IA. Inspeção técnica: src/negocio/Main.java, GerenciadoraClientes.java, GerenciadoraContas.java, Cliente.java, ContaCorrente.java e testes RF02/RF03/RF09/RF08/RF12; .classpath.

Documentação oficial consultada em 26/10/2026

[1] **JaCoCo - Maven Plug-in**
https://www.jacoco.org/jacoco/trunk/doc/maven.html

[2] **PIT - Maven Quick Start**
https://pitest.org/quickstart/maven/

[3] **Cucumber-JVM - Installation**
https://cucumber.io/docs/installation/java/

[4] **Spring Framework - MockMvc**
https://docs.spring.io/spring/reference/testing/mockmvc.html

[5] **Cypress - Writing your first end-to-end test**
https://docs.cypress.io/app/end-to-end-testing/writing-your-first-end-to-end-test

Antes de instalar, conferir compatibilidade e fixar versões estáveis. As metas de cobertura, contratos HTTP e desenho técnico deste roteiro são propostas do projeto; não representam resultados dos artigos nem requisitos adicionais impostos pelo professor.